WEEK 5
PAYMENT TRANSACTION MANAGEMENT SYSTEM

1. Introduction:

A Payment Transaction Management System is a database application used to store and manage payment details related to online orders. In an e-commerce business, customers use different payment methods such as UPI, debit or credit cards, net banking, and cash.

This system maintains payment information, monitors transaction status, identifies successful and failed payments, and prepares reports about revenue and customer payment history. It uses SQL queries to store, retrieve, and analyze payment records efficiently.

2. Aim of the Project

The main aim of this project is to develop a database system that records customer payment transactions and generates useful reports about payment status, payment methods, and revenue.

3. Objectives

- To create a database table for storing payment information.
- To maintain payment details for different orders.
- To identify successful and failed transactions.
- To calculate the number of transactions for each payment method.
- To calculate total revenue and average transaction amount.
- To display customer payment history by connecting customer, order, and payment tables.
- To generate reports that help analyze payment activities.

5. Database Design

The Payment table contains the following attributes:

Attribute	Description
Payment_ID	Unique number for each payment
Order_ID	Reference to the related order
Payment_Date	Date of the transaction
Payment_Mode	Method used to make the payment
Payment_Status	Current transaction status
Transaction_Amount	Amount recorded for the payment

6. SQL Queries and Implementation

Query 1: Create the Payment Table

This query creates a table to store payment transaction details

```
SQL> CREATE TABLE Payment (
  2      Payment_ID NUMBER PRIMARY KEY,
  3      Order_ID NUMBER,
  4      Payment_Date DATE,
  5      Payment_Mode VARCHAR2(20),
  6      Payment_Status VARCHAR2(20),
  7      Transaction_Amount NUMBER(10,2)
  8  );

Table created.
```

The table stores the payment ID, order reference, date, payment method, transaction status, and amount.

7.Insert Payment Records:

The following queries insert 15 sample payment records.

```
SQL> INSERT INTO Payment VALUES (107, 1007, DATE '2026-10-07', 'UPI', 'Success', 2200);

1 row created.

SQL> INSERT INTO Payment VALUES (108, 1008, DATE '2026-10-08', 'Card', 'Success', 4500);

1 row created.

SQL> INSERT INTO Payment VALUES (109, 1009, DATE '2026-10-09', 'Net Banking', 'Failed', 1700);

1 row created.

SQL> INSERT INTO Payment VALUES (110, 1010, DATE '2026-10-10', 'UPI', 'Success', 3200);

1 row created.

SQL> INSERT INTO Payment VALUES (111, 1011, DATE '2026-10-11', 'Cash', 'Success', 900);

1 row created.

SQL> INSERT INTO Payment VALUES (112, 1012, DATE '2026-10-12', 'Card', 'Pending', 2800);

1 row created.

SQL> INSERT INTO Payment VALUES (113, 1013, DATE '2026-10-13', 'UPI', 'Failed', 1100);

1 row created.

SQL> INSERT INTO Payment VALUES (114, 1014, DATE '2026-10-14', 'Net Banking', 'Success', 5000);

1 row created.

SQL> INSERT INTO Payment VALUES (115, 1015, DATE '2026-10-15', 'Card', 'Success', 3500);

1 row created.
```

These records provide sample data for testing the queries and generating payment reports. The order IDs are set to 1–10 to match the example Orders table; confirm that those IDs exist in your database.

8.Display All Payment Records :

```
SQL> SELECT*FROM PAYMENT;

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       101       1001 01-OCT-26 UPI                  Success
              1500

       102       1002 02-OCT-26 Card                 Success
              2500

       103       1003 03-OCT-26 UPI                  Failed
              1200


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       104       1004 04-OCT-26 Net Banking          Success
              3000

       105       1005 05-OCT-26 Card                 Failed
              1800

       106       1006 06-OCT-26 Cash                 Pending
               800


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       107       1007 07-OCT-26 UPI                  Success
              2200

       108       1008 08-OCT-26 Card                 Success
              4500

       109       1009 09-OCT-26 Net Banking          Failed
              1700


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       110       1010 10-OCT-26 UPI                  Success
              3200

       111       1011 11-OCT-26 Cash                 Success
               900

       112       1012 12-OCT-26 Card                 Pending
              2800


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       113       1013 13-OCT-26 UPI                  Failed
              1100

       114       1014 14-OCT-26 Net Banking          Success
              5000

       115       1015 15-OCT-26 Card                 Success
              3500


15 rows selected.
```

This query displays every payment record stored in the table.

9.Display Successful Payments:

```
SQL> SELECT *
  2  FROM Payment
  3  WHERE Payment_Status = 'Success';

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       101       1001 01-OCT-26 UPI                  Success
              1500

       102       1002 02-OCT-26 Card                 Success
              2500

       104       1004 04-OCT-26 Net Banking          Success
              3000


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       107       1007 07-OCT-26 UPI                  Success
              2200

       108       1008 08-OCT-26 Card                 Success
              4500

       110       1010 10-OCT-26 UPI                  Success
              3200


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       111       1011 11-OCT-26 Cash                 Success
               900

       114       1014 14-OCT-26 Net Banking          Success
              5000

       115       1015 15-OCT-26 Card                 Success
              3500


9 rows selected.
```

This query retrieves transactions whose status is marked as successful.

10.Find Failed Transactions:

```
SQL> SELECT *
  2  FROM Payment
  3  WHERE Payment_Status = 'Failed';

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       103       1003 03-OCT-26 UPI                  Failed
              1200

       105       1005 05-OCT-26 Card                 Failed
              1800

       109       1009 09-OCT-26 Net Banking          Failed
              1700


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       113       1013 13-OCT-26 UPI                  Failed
              1100
```

This query identifies payments that were unsuccessful.

11.Count Successful and Failed Payments:

```
SQL> SELECT Payment_Status, COUNT(*) AS Total
  2  FROM Payment
  3  WHERE Payment_Status IN ('Success', 'Failed')
  4  GROUP BY Payment_Status;

PAYMENT_STATUS            TOTAL
-------------------- ----------
Failed                         4
Success                        9
```

This query calculates the number of successful and failed transactions separately.

12.Payment Mode Analysis:

This section studies how frequently customers use different payment methods.

Count UPI Transactions:

```
SQL> SELECT COUNT(*) AS UPI_Count
  2  FROM Payment
  3  WHERE Payment_Mode = 'UPI';

 UPI_COUNT
----------
         5

SQL>
```

This query returns the total number of UPI payment records.

Count Card Payments:

```
SQL> SELECT COUNT(*) AS Card_Count
  2  FROM Payment
  3  WHERE Payment_Mode = 'Card';

CARD_COUNT
----------
         5
```

This query counts the transactions made using cards.

13.Find the Most Preferred Payment Method:

```
SQL> SELECT Payment_Mode, COUNT(*) AS Total
  2  FROM Payment
  3  GROUP BY Payment_Mode
  4  ORDER BY Total DESC;

PAYMENT_MODE              TOTAL
-------------------- ----------
Card                          5
UPI                           5
Net Banking                   3
Cash                          2
```

The query groups transactions by payment method and arranges them from the most frequently used to the least frequently used. The first row indicates the most-used method. If methods have equal counts, they share the top position.

14.Revenue Analysis:

Revenue analysis helps determine the amount received through successful transactions.

Calculate Total Revenue:

```
SQL> SELECT SUM(Transaction_Amount) AS Total_Revenue
  2  FROM Payment
  3  WHERE Payment_Status = 'Success';

TOTAL_REVENUE
-------------
        26300
```

This query adds the amounts of all successful transactions. Failed and pending payments are excluded.

15.Calculate Revenue by Payment Method:

```
SQL> SELECT Payment_Mode,
  2         SUM(Transaction_Amount) AS Revenue
  3  FROM Payment
  4  WHERE Payment_Status = 'Success'
  5  GROUP BY Payment_Mode;

PAYMENT_MODE            REVENUE
-------------------- ----------
Net Banking                8000
Card                      10500
Cash                        900
UPI                        6900
```

This query calculates the total successful payment amount for each payment method.

16.Calculate Average Transaction Amount:

```
SQL> SELECT AVG(Transaction_Amount) AS Average_Amount
  2  FROM Payment
  3  WHERE Payment_Status = 'Success';

AVERAGE_AMOUNT
--------------
    2922.22222
```

This query calculates the average amount of successful payments.

17.Customer Payment History:

Customer payment history combines information from the Customer, Orders, and Payment tables. It displays the customer's name, order number, payment method, amount, and transaction status.

Display Customer Payment History:

```
SQL> SELECT C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  2         O.Order_ID,
  3         P.Payment_Mode,
  4         P.Transaction_Amount,
  5         P.Payment_Status
  6  FROM Customer C
  7  JOIN Orders O
  8  ON C.Customer_ID = O.Customer_ID
  9  JOIN Payment P
 10  ON O.Order_ID = P.Order_ID;

CUSTOMER_NAME
--------------------------------------------------------------------------------
ORDER_I PAYMENT_MODE         TRANSACTION_AMOUNT PAYMENT_STATUS
------- -------------------- ------------------ --------------------
Arun Kumar
0001    UPI                                1500 Success

Meena Ravi
0002    Card                               2500 Success

Karthik Raj
0003    UPI                                1200 Failed


CUSTOMER_NAME
--------------------------------------------------------------------------------
ORDER_I PAYMENT_MODE         TRANSACTION_AMOUNT PAYMENT_STATUS
------- -------------------- ------------------ --------------------
Swetha Mohan
0004    Net Banking                        3000 Success

Imran Ali
0005    Card                               1800 Failed

Keerthi Suresh
0006    Cash                                800 Pending


CUSTOMER_NAME
--------------------------------------------------------------------------------
ORDER_I PAYMENT_MODE         TRANSACTION_AMOUNT PAYMENT_STATUS
------- -------------------- ------------------ --------------------
Rohan Sharma
0007    UPI                                2200 Success

Pooja Nair
0008    Card                               4500 Success

Sanjay Das
0009    Net Banking                        1700 Failed


CUSTOMER_NAME
--------------------------------------------------------------------------------
ORDER_I PAYMENT_MODE         TRANSACTION_AMOUNT PAYMENT_STATUS
------- -------------------- ------------------ --------------------
Harini Krishnan
0010    UPI                                3200 Success


10 rows selected.
```

The query connects the three tables through their related IDs and displays the payment details alongside the customer's name.

18.Overall Payment Analysis:

Generate a Payment Status Summary:

```
SQL> SELECT Payment_Status,
  2         COUNT(*) AS Total_Transactions,
  3         SUM(Transaction_Amount) AS Total_Amount
  4  FROM Payment
  5  GROUP BY Payment_Status;

PAYMENT_STATUS       TOTAL_TRANSACTIONS TOTAL_AMOUNT
-------------------- ------------------ ------------
Failed                                4         5800
Pending                               2         3600
Success                               9        26300
```

19.Conclusion:

The Payment Transaction Management System demonstrates how SQL can be used to maintain and analyze payment records in an e-commerce database. The project supports payment data storage, transaction status checking, payment method analysis, revenue calculations, and customer payment history reporting.

By connecting payment information with existing customer and order records, the system provides a more complete view of transaction activity. It also demonstrates the practical use of SQL commands such as `CREATE TABLE`, `INSERT`, `SELECT`, `WHERE`, `GROUP BY`, `ORDER BY`, aggregate functions, and table joins.

Overall, the project provides a foundation for managing payment information accurately and generating reports that support business analysis.